Analysis and Design of Algorithms

CS3230

Recitation

Week 3

Prof Lee Wee Sun

Recursive Algorithm Runtime

In typical recursive or divide and conquer algorithms, the total runtime is the time to solve the subproblems recursively and the time spent within the procedure.

Can write as a **recurrence**. Most common form is in the form:

$$T(n) = aT(n/b) + \Theta(f(n))$$

To solve a recurrence, we can

- Use a recursion tree
- Use substitution method (mathematical induction)
- Use Master method

Master Theorem

$$T(n) = aT(n/b) + \Theta(f(n))$$

Case 1:

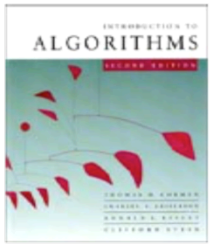
$$f(n) = O(n^{\log_b a - \epsilon})$$
$$T(n) = \Theta(n^{\log_b a})$$

Case 2:

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$
$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Case 3:

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$
$$af(n/b) \leq cf(n), c < 1$$
$$T(n) = \Theta(f(n))$$



Idea of master theorem

Recursion tree:

$$
\begin{array}{lcccccc}
 & f(n) & & & & & f(n) \\
 & & a & & & & \\
 & f(n/b) & f(n/b) & \cdots & f(n/b) & & a\,f(n/b) \\
h = \log_b n & & a & & & & \\
 & f(n/b^2) & f(n/b^2) & \cdots & f(n/b^2) & & a^2 f(n/b^2) \\
 & \vdots & & & & & \vdots \\
 & T(1) & & & & & n^{\log_b a}\,T(1)
\end{array}
$$

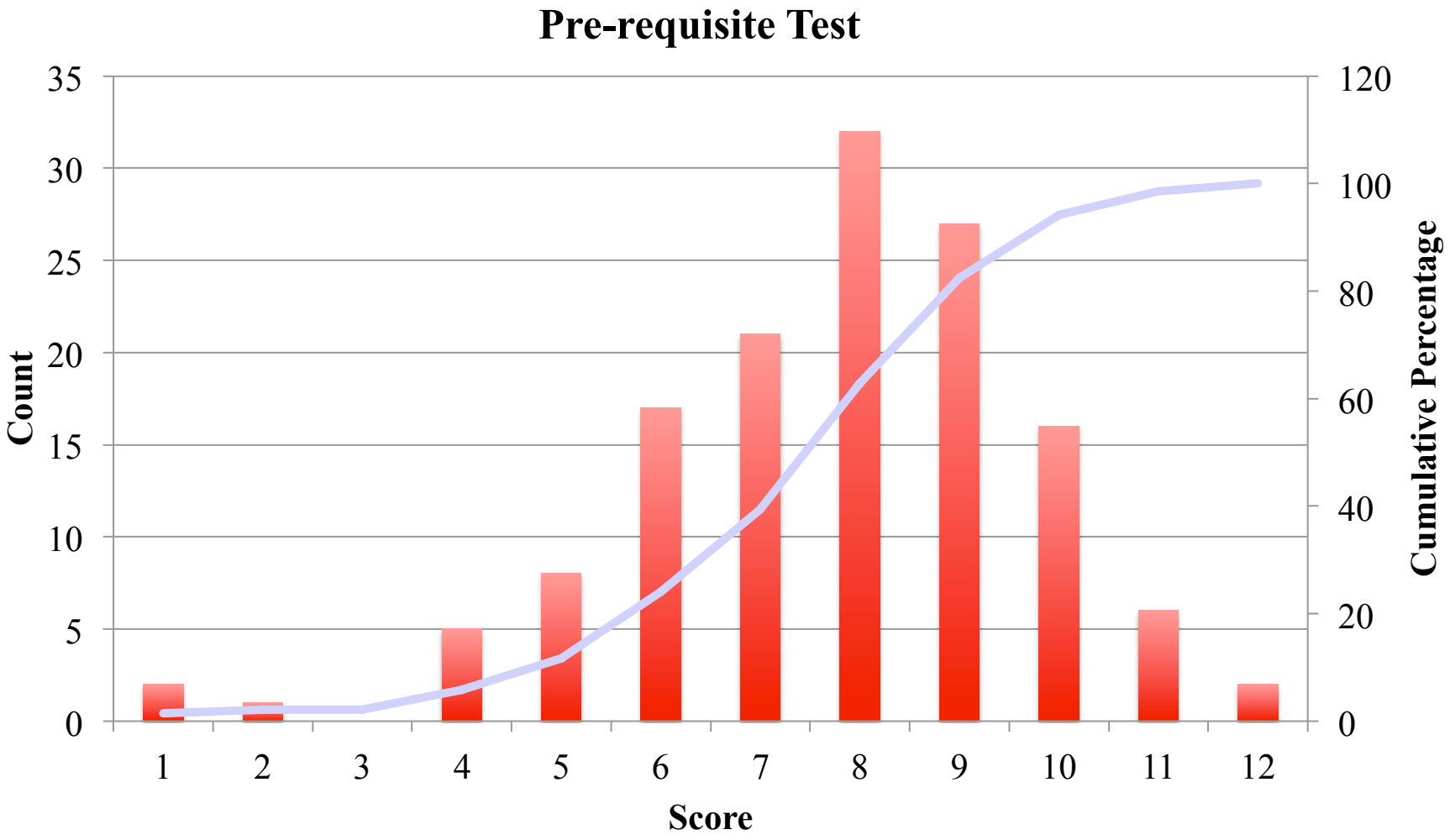
$$
\begin{aligned}
\#\text{leaves} &= a^h \\
&= a^{\log_b n} \\
&= n^{\log_b a}
\end{aligned}
$$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Admin

- Homework 1 out. Due Friday 9 Sept. Submit file through Coursemology. You should be able to do, after watching this week's video.
- Side quest training released last week. Harder questions than those we do in recitations and tutorials.
- Tutorials start this week. Finalize your tutorials by noon today. After Yang Yang updates, I'll allocate to the latest on CORS (may not be synchronized with IVLE).

Pre-req Test


Pre-requisite Test
Count
Cumulative Percentage
Score
35
30
25
20
15
10
5
0
120
100
80
60
40
20
0
1
2
3
4
5
6
7
8
9
10
11
12

Pre-req test Q10

We would like to implement a dynamic set with insert, delete and search functions to be used for a real-time system. The requirements are:

1. As it is a real-time system, each operation **must** be completed within a fixed time limit, e.g. 1 millisecond.
2. The maximum size of the set is quite large so the operations need to be efficient.

Which of the following data structure would be the most appropriate?

- Hash table
- Linked list
- Balanced tree
- Array

Average case runtime O(1)

Worst case runtime O(lg n)

Pre-req Test Q12

The bus service has a new way of computing fare that depends not only on the start and ending point of a trip, but also on whether the passenger has switch from one bus to another during the journey. To do that, it decided to charge passengers as follows: the cost of travelling on a bus from A to B depends not only on the pair (A,B) but also on the passenger's previous location before A (where the previous location may be a special start node if A is the starting location of the trip), i.e. $w(C,A,B)$ is the cost of travelling on the bus from A to B when the passenger previously travelled on another bus from C to get to A and $w(START,A,B)$ is the cost of travelling on the bus from A to B when A is the starting point of the journey. Select the **fastest correct** algorithm to find the cheapest path from one location to another from the following choices.

Modify Dijkstra's algorithm (call it modified Dijkstra's algorithm) as follows:

When a vertex u is extracted from the priority queue, for each neighbour v, if $d[v] > d[u] + w(\pi[u], u, v)$ then

- update $d[v] = d[u] + w(\pi[u], u, v)$

- update the parent pointer $\pi(v)$ to u

where $w(\pi[u], u, v)$ is the cost of traveling from u to v when the previous edge travelled is $(\pi(u), u)$.

Weight changes depending on how we got to u.
Dijkstra does not apply. See solution for counterexample.

We create a graph as follows:

- For each pair of locations (x, y) create a vertex v_{xy}.

- The cost of the edge (v_{xy}, v_{yz}) is the cost of traveling from y to z when the previous edge travelled is (x, y).

Then run Dijkstra's algorithm on the graph to find the cheapest path.

In the new graph, edge weight constant, so Dijkstra applies.

MCQ

Question 1

Which of these relationships is **incorrect**?

- $n = O(n^2)$
- $n = \Theta(n^2)$
- $n = \Omega(\sqrt{n})$
- $n^2 = \Omega(n \log n)$

Solution

$n = O(n^2)$

- **True** as n grows slower than n^2

$n = \Theta(n^2)$

- **False** as n grows slower than n^2

$n = \Omega(\sqrt{n})$

- **True** as $\sqrt{n}$ grows slower than n

$n = \Omega(n \log n)$

- **True** as $\log n$ grows slower than n

- $f(n) = O(g(n))$ if there exist constants $c > 0$, $n_0 > 0$ such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$
- $f(n) = \Omega(g(n))$ if there exist constants $c > 0$, $n_0 > 0$ such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0$
- $f(n) = \Theta(n)$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$

Question 2

Write and solve the recurrence for RUNTIME(n)

```
RUNTIME(n)
if n = 1 return
else
    for i = 1 to n
        print 'Running …'
    for i = 1 to 3
        RUNTIME(n/3)
```

- $T(n) = T(n/3) + \Theta(n)$
  $T(n) = \Theta(n)$
- $T(n) = 3T(n/3) + \Theta(n)$
  $T(n) = \Theta(n \lg n)$
- $T(n) = 3T(n/3) + \Theta(1)$
  $T(n) = \Theta(n \lg n)$
- $T(n) = 3T(n/3) + \Theta(n)$
  $T(n) = \Theta(n)$

Solution

$$T(n) = 3T(n/3) + \Theta(n)$$

$$f(n) = \Theta(n)$$

$$= \Theta(n^{\log_3 3} \log^0 n)$$

Case 2:

$$T(n) = \Theta(n \log n)$$

Case 2

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

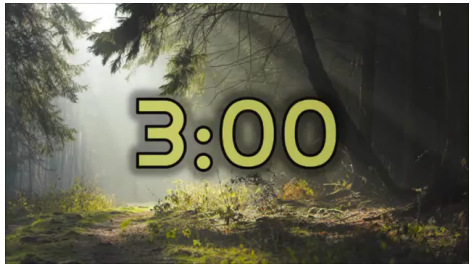
$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Question 3

Write and solve the recurrence for RUNTIME(n)

```
RUNTIME(n)
if n = 1 return
else
    for i = 1 to log n
        print 'Running …'
    for i = 1 to 2
        RUNTIME(n/10)
```

- $T(n) = T(n/10) + \Theta(n)$
  $T(n) = \Theta(n)$
- $T(n) = T(n/10) + \Theta(\lg n)$
  $T(n) = \Theta(\lg^2 n)$
- $T(n) = 2T(n/10) + \Theta(\lg n)$
  $T(n) = \Theta(n^{\log_{10} 2})$
- $T(n) = 2T(n/10) + \Theta(n)$
  $T(n) = \Theta(n)$



Solution

$$T(n) = 2T(n/10) + \Theta(\log n)$$

$$f(n) = \Theta(\log n)$$

$$= O(n^{\log_{10} 2 - \epsilon})$$

where $\log_{10} 2 \approx 0.3$

Fact:

$\log n = O(n^{\alpha})$

for any positive α

Case 1:

$$T(n) = \Theta(n^{\log_{10} 2})$$

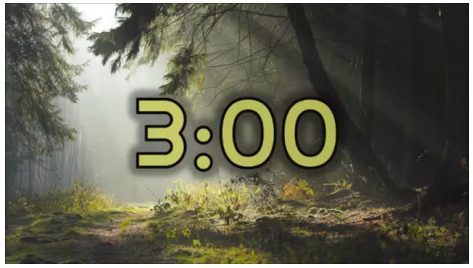
Case 1	$f(n) = O(n^{\log_b a - \epsilon})$ $T(n) = \Theta(n^{\log_b a})$

Question 4

Write and solve the recurrence for RUNTIME(n)

```
RUNTIME(n)
if n = 1 return
else
    for i = 1 to n log n!
        print 'Running …'
    for i = 1 to 9
        RUNTIME(n/3)
```

- $T(n) = 9T(n/3) + \Theta(n \lg n!)$
  $T(n) = \Theta(n^2)$
- $T(n) = 9T(n/3) + \Theta(n \lg n!)$
  $T(n) = \Theta(n^2 \log^2 n)$
- $T(n) = 9T(n/3) + \Theta(n \lg n)$
  $T(n) = \Theta(n^2)$
- $T(n) = T(n/3) + \Theta(n \lg n!)$
  $T(n) = \Theta(n^2)$



Solution

$$T(n) = 9T(n/3) + \Theta(n \log n!)$$

$$f(n) = \Theta(n \log n!)$$

$$= \Theta(n^2 \log n)$$

$$= \Theta(n^{\log_3 9} \log^1 n)$$

Fact:

$\log n! = \Theta(n \log n)$

Case 2:

$$T(n) = \Theta(n^2 \log^2 n)$$

Case 2

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Question 5

Write and solve the recurrence for RUNTIME(n)

RUNTIME(n)

if $n = 1$ **return**

else

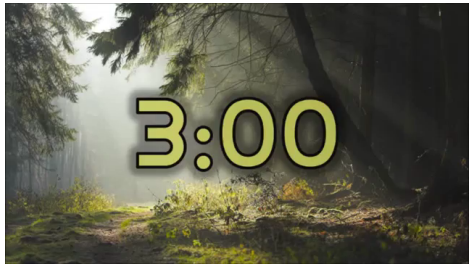
  for $i = 1$ **to** $2^{\lg n}$

    print 'Running …'

  for $i = 1$ **to** 3

    RUNTIME($n/3$)

- $T(n) = T(n/3) + \Theta(2^{\lg n})$
  $T(n) = \Theta(2^{\lg n})$
- $T(n) = 3T(n/3) + \Theta(n)$
  $T(n) = \Theta(n \lg n)$
- $T(n) = 3T(n/3) + \Theta(n \lg n)$
  $T(n) = \Theta(n \lg^2 n)$
- $T(n) = 3T(n/3) + \Theta(n)$
  $T(n) = \Theta(n)$



Solution

$$T(n) = 3T(n/3) + \Theta(n)$$
$$f(n) = \Theta(n)$$
$$= \Theta(n^{\log_3 3} \log^0 n)$$

Fact:

$$2^{\lg n} = n$$

Case 2:

$$T(n) = \Theta(n \log n)$$

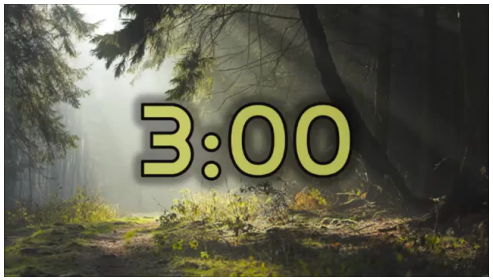
Case 2

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$
$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Question 6

What is the asymptotic runtime for running `meanFunction(n)`?

- $\Theta(1)$
- $\Theta(n)$
- $\Theta(n \log n)$
- $\Theta(n^2)$



```
int niceFunction(int n){
    for    (int i = 0; i < n; i++){
        System.out.println("I am nice!");
    }
    return 42;
}
int meanFunction(int n){
    if(n == 0) return 0;
    return 2 * meanFunction(n/2) + niceFunction(n);
```

Solution

$$T(n) = T(n/2) + \Theta(n)$$

$$f(n) = \Theta(n) = \Omega(n^{\log_2 1+\epsilon})$$

Reg. Cond:

$$f(n/2) = n/2 = f(n)/2$$

$$\text{Case 3} : T(n) = \Theta(n)$$

Case 3	$f(n) = \Omega(n^{\log_b a+\epsilon})$ $af(n/b) \leq cf(n), c < 1$ $T(n) = \Theta(f(n))$

Alternate solution with last line

return meanFunction(n/2)*meanFunction(n/2) + niceFunction(n)

gives $T(n) = 2T(n/2) + \Theta(n)$ with runtime of $\Theta(n \log n)$.

Analysis tells us that we can get a better design with runtime $\Theta(n)$ by doing only one call to meanFunction(n/2).

Question 7

Who is the Master of Algorithms pictured below?

- Alan Turing
- Richard Karp
- Robert Floyd
- Donald Knuth

Solution

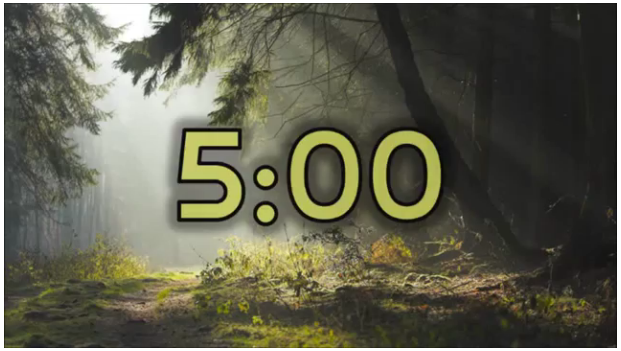
Donald Knuth

Turing Award winner
Author of *The Art of Computer Programming*

Has been called “father of analysis of algorithms”. Popularized asymptotic notation. Creator of TeX typesetting system.

Structured Question

Part (a)




```
FIB(n)
if n == 0
    return 0
elseif n == 1
    return 1
return FIB(n - 1) + FIB(n - 2)
```

Give a recurrence for $T(n)$. Then use the substitution method to show that $T(n) = O(F_n)$.

Solution

$T(n) = T(n - 1) + T(n - 2) + a$

Inductive hypothesis: $T(k) \leq bF_k - c$ for $k < n$.

$T(n) = T(n - 1) + T(n - 2) + a$

$\leq bF_{n\text{-}1} - c + bF_{n\text{-}2} - c + a$

$= b(F_{n\text{-}1} + F_{n\text{-}2}) - \mathrm{c} - (\mathrm{c} - \mathrm{a})$

$= bF_n - c - (c - a)$

$\leq bF_n - c$

for $c \geq a$. Possible to select b large enough for base cases $n = 1$ and $n = 2$ to hold.

Common Errors

- Use inductive hypothesis $T(k) \leq bF_k$ and fail to prove the inductive step.
- Use wrong recurrence:
  $T(n) = T(n - 1) + T(n - 2)$
- In inductive step do, assume $T(k) = O(F_k)$ for all $k < n$, and

$$T(n) = T(n - 1) + T(n - 2) + a$$

$$\leq bF_{n-1} + bF_{n-2} + a$$

$$= bF_n + a \ = O(F_n)$$

Part (b)

Use the substitution method to show that $T(n) = \Omega(F_n)$ and hence $T(n) = \Theta(F_n)$.

Solution

$T(n) = T(n-1) + T(n-2) + a$
for some constant a.
Assume as inductive hypothesis $T(k) \geq b' F_k$
for all $k < n$. Then,

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + a \\ &\geq b' F_{n-1} + b' F_{n-2} + a \\ &= b' F_n + a \\ &\geq b' F_n \end{aligned}$$

Similarly can select b small enough for base cases $n = 1$ and $n = 2$.

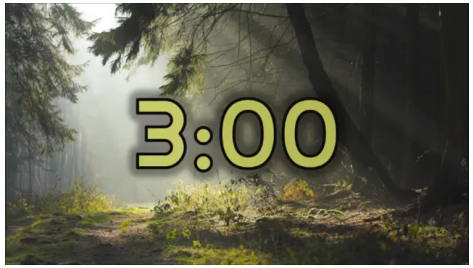
Part (c)

FIB'(n)

1. **if** n == 0
2. **return** 0
3. **elseif** n == 1
4. **return** 1
5. sum = 1
6. **for** k = 1 to n - 2
7. sum = sum + FIB'(k)
8. **return** sum

First, assume FIB'(k) correctly returns the k-th Fibonacci number for all $k < n$.

State a suitable invariant for the loop on line 6



Solution

for $k = 1$ to $n - 2$

$\quad sum = sum + \text{FIB'}(k)$

A suitable invariant is

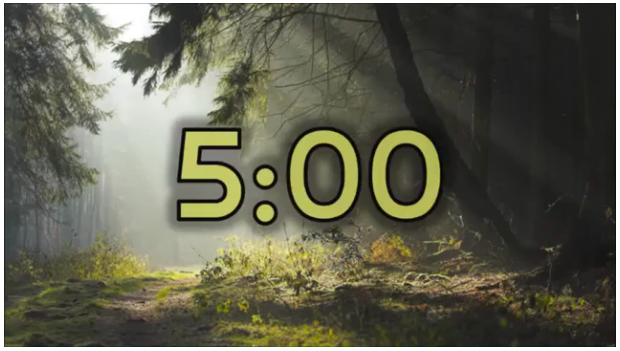
$sum = F_{k+1}$

Why not F_k or F_{k+2}?

Part (d)

Assume FIB'(k) correctly returns the k-th Fibonacci number for all $k < n$.

Prove the invariant, then argue correctness of FIB'(n).



Solution

sum = 1
for *k* = 1 **to** *n* - 2
    sum = *sum* + FIB'(*k*)

Invariant

$$\text{sum} = F_{k+1}$$

Initialization:

sum = 1, $k = 1$,

Before entering loop, sum = 1 = F_2 as required.

sum = 1
for *k* = 1 **to** *n* - 2
  sum = *sum* + FIB'(*k*)

Invariant

$\text{sum} = F_{k+1}$

Maintenance: Assume invariant true at start of iteration. Show invariant true at the start of next iteration.

Within iteration variable *sum* becomes

$sum' = sum + \text{FIB}'(k) = F_{k+1} + F_k = F_{k+2}$

k becomes $k' = k+1$, so $sum' = F_{k+2} = F_{k'+1}$

sum = 1

for k = 1 **to** n - 2

sum = sum + FIB'(k)

Invariant

$$\text{sum} = F_{k+1}$$

Termination: $k = n$-1, so *sum* = F_n

Together with assumption that FIB'(k) = F_k (inductive hypothesis for proof of recursive algorithm) we have shown that FIB'(n) = F_n for all n.

```
FIB'(n)
if n == 0
    return 0
elseif n == 1
    return 1
sum = 1
for k = 1 to n - 2
    sum = sum + FIB'(k)
return sum
```

To prove FIB'(n), correct:

Base case: F_0 and F_1 correct by definition of Fib number

Inductive step: Assume FIB'(k) correct for $k < n$. Invariant shows FIB'(n) returns F_n upon termination.